Python Technical MCQ — Round 2

Answer Key | 40 Questions | For 2 Years Experience | Duration: 30 min

CONFIDENTIAL — Interviewer Use Only

Test Overview

Total Questions: 40 | Categories: Functions & Scope, Strings, Lists & Tuples, Dictionaries & Sets, Decorators & Closures, OOP, Exception Handling, Comprehensions & Generators, Data Types, Core Python

Difficulty: Intermediate (2 Years Experience) | Topics: Core Python, Data Types, Collections, Functions, OOP, Generators, Decorators

Quick Answer Reference

Q1: B

Q2: B

Q3: B

Q4: B

Q5: B

Q6: B

Q7: B

Q8: B

Q9: A

Q10: B

Q11: B

Q12: B

Q13: C

Q14: A

Q15: B

Q16: B

Q17: A

Q18: B

Q19: B

Q20: B

Q21: A

Q22: B

Q23: B

Q24: B

Q25: B

Q26: A

Q27: A

Q28: B

Q29: A

Q30: B

Q31: A

Q32: A

Q33: B

Q34: B

Q35: B

Q36: B

Q37: B

Q38: B

Q39: B

Q40: B

Detailed Questions & Answers

FUNCTIONS & SCOPE

Q1. What is the output of the following code?

```
def add(item, lst=[]):
    lst.append(item)
    return lst

print(add(1))
print(add(2))
```

A. [1]
[2]

B. [1]
[1, 2] '

C. [1, 2]
[1, 2]

D. Error

Explanation: Default mutable arguments are evaluated once at function definition and shared across calls. The same list persists, so the second call appends to the list that already contains 1, producing [1, 2].

STRINGS

Q2. What is the output of the following code?

```
s = "Python"
s.replace("P", "J")
print(s)
```

A. Jython

B. Python '

C.

D. Error

Explanation: Strings are immutable. str.replace() returns a new string but does not modify the original. Since the return value is discarded, s still refers to "Python".

LISTS & TUPLES

Q3. What is the output of the following code?

```
lst = [1, 2, 3, 4, 5]
print(lst[::-1])
```

A. [1, 2, 3, 4, 5]

B. [5, 4, 3, 2, 1] '

C. [5, 4, 3, 2, 1, 0]

D. Error

Explanation: The slice [::-1] uses a step of -1, which traverses the list from end to start, returning a reversed copy.

LISTS & TUPLES

Q4. What is the output of the following code?

```
matrix = [[0] * 3] * 2
matrix[0][0] = 1
print(matrix)
```

A. [[1, 0, 0], [0, 0, 0]]

B. [[1, 0, 0], [1, 0, 0]] '

C. [[1, 1, 1], [1, 1, 1]]

D. Error

Explanation: [[0] * 3] * 2 creates a list with two references to the SAME inner list. Mutating matrix[0][0] changes the shared inner list, so both rows show the change.

DICTIONARIES & SETS

Q5. What is the output of the following code?

```
d = {'a': 1, 'b': 2}
print(d.get('c', 0))
```

A. None

B. 0 '

C. KeyError

D. ''

Explanation: dict.get(key, default) returns the value for the key if present, otherwise the supplied default. 'c' is absent, so the default 0 is returned (no exception).

DICTIONARIES & SETS

Q6. What is the output of the following code?

```
s = {1, 2, 2, 3, 3, 3}
print(len(s))
```

A. 6

B. 3 '

C. 1

D. Error

Explanation: A set stores only unique elements. Duplicate values are discarded, leaving {1, 2, 3}, so the length is 3.

LISTS & TUPLES

Q7. What is the output of the following code?

```
t = (5)
print(type(t).__name__)
```

A. tuple

B. int '

C. list

D. Error

Explanation: (5) is just the integer 5 in parentheses, not a tuple. A single-element tuple requires a trailing comma: (5,).

FUNCTIONS & SCOPE

Q8. What is the output of the following code?

```
def f(*args, **kwargs):
    return len(args), len(kwargs)

print(f(1, 2, a=3))
```

A. (3, 0)

B. (2, 1) '

C. (1, 2)

D. Error

Explanation: Positional arguments 1 and 2 are collected into args (length 2). The keyword argument a=3 goes into kwargs (length 1). So the result is (2, 1).

FUNCTIONS & SCOPE

Q9. What is the output of the following code?

```
x = 10
def f():
    x = 20
f()
print(x)
```

A. 10 '

B. 20

C. None

D. Error

Explanation: Assigning to x inside f() creates a new local variable, leaving the global x untouched. After the call, the global x is still 10.

FUNCTIONS & SCOPE

Q10. What is the output of the following code?

```
x = 10
def f():
    global x
    x = 20
f()
print(x)
```

A. 10

B. 20 '

C. Error

D. None

Explanation: The global statement makes x refer to the module-level variable, so the assignment inside f() rebinds the global x to 20.

DECORATORS & CLOSURES

Q11. What is the output of the following code?

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])
```

A. [0, 1, 2]

B. [2, 2, 2] '

C. [0, 0, 0]

D. Error

Explanation: Closures capture variables by reference, not by value (late binding). By the time the lambdas are called, the loop variable i is 2, so every lambda returns 2.

OOP

Q12. What is the output of the following code?

```
class A:
    def __init__(self):
        self.x = 1

class B(A):
    def __init__(self):
        self.y = 2

b = B()
print(hasattr(b, 'x'))
```

A. True

B. False '

C. Error

D. None

Explanation: B.__init__ overrides A.__init__ and never calls super().__init__(), so self.x is never set. hasattr(b, 'x') is therefore False.

OOP

Q13. What is the output of the following code?

```
class Counter:
    count = 0
    def __init__(self):
        Counter.count += 1

a = Counter()
b = Counter()
```

```
print(Counter.count)
```

A. 0

B. 1

C. 2 '

D. Error

> Explanation: count is a class variable. Each instantiation increments Counter.count, so after creating two instances it equals 2.

OOP

Q14. What is the output of the following code?

```
class P:
    def __str__(self):
        return "P obj"

print(P())
```

A. P obj '

B. <__main__.P object>

C. Error

D. <P>

> Explanation: print() calls str() on the object, which invokes the __str__ method. The custom __str__ returns "P obj".

EXCEPTION HANDLING

Q15. What is the output of the following code?

```
try:
    x = 1
except:
    print("except")
else:
    print("else")
```

A. except

B. else '

C. except
else

D. (nothing)

> Explanation: The else block of a try statement runs only when no exception is raised in the try block. No error occurs, so "else" is printed.

EXCEPTION HANDLING

Q16. What is the output of the following code?

```
def f():
    try:
        return 1
    finally:
        return 2

print(f())
```

A. 1

B. 2 '

C. Error

D. None

Explanation: A return in the finally block overrides any return from the try block. The function returns 2.

COMPREHENSIONS & GENERATORS

Q17. What is the output of the following code?

```
print({x: x**2 for x in range(3)})
```

A. {0: 0, 1: 1, 2: 4} '

B. {1: 1, 2: 4, 3: 9}

C. [0, 1, 4]

D. Error

Explanation: This dict comprehension maps each x in range(3) (0, 1, 2) to its square, producing {0: 0, 1: 1, 2: 4}.

COMPREHENSIONS & GENERATORS

Q18. What is the output of the following code?

```
def gen():
    yield 1

g = gen()
print(next(g))
print(next(g))
```

A. 1, then None

B. 1, then a StopIteration exception '

C. 1, then 1

D. 1, then 2

Explanation: The generator yields 1 once. The second next() call finds no more values and raises StopIteration.

DECORATORS & CLOSURES

Q19. What is the output of the following code?

```
def deco(f):
    def wrapper():
        return f() * 2
    return wrapper

@deco
def num():
    return 5

print(num())
```

A. 5

B. 10 '

C. Error

D. 25

Explanation: The @deco decorator replaces num with wrapper, which calls the original num() (returning 5) and multiplies by 2, giving 10.

DATA TYPES

Q20. What is the output of the following code?

```
print(True + True + False)
```

A. Error

B. 2 '

C. True

D. 1

Explanation: bool is a subclass of int, where True == 1 and False == 0. So True + True + False evaluates to 1 + 1 + 0 = 2.

DATA TYPES

Q21. What is the output of the following code?

```
print(7 // 2)
print(7 / 2)
```

A. 3
3.5 '
B. 3.5
3.5
C. 3
3
D. 3.5
3

Explanation: // is floor (integer) division returning 3, while / is true division returning the float 3.5.

STRINGS

Q22. What is the output of the following code?

```
x = 5
print(f"{x * 2}")
```

A. {x * 2}

B. 10 '

C. 5 * 2

D. Error

Explanation: An f-string evaluates the expression inside the braces. x * 2 is 10, which is inserted into the string.

LISTS & TUPLES

Q23. What is the output of the following code?

```
a = [1, 2]
a.append([3, 4])
print(len(a))
```

A. 4

B. 3 '

C. 2

D. Error

Explanation: append() adds its argument as a single element. The list [3, 4] becomes one element, so a is [1, 2, [3, 4]] with length 3. (extend() would have given length 4.)

DICTIONARIES & SETS

Q24. What is the output of the following code?

```
d = {'a': 1}
d['b'] = 2
d['a'] = 3
print(d)
```

A. {'a': 1, 'b': 2}

B. {'a': 3, 'b': 2} '

C. {'b': 2, 'a': 3}

D. Error

Explanation: Assigning to an existing key updates its value in place without changing insertion order. 'a' stays first with the updated value 3, then 'b': 2.

CORE PYTHON

Q25. What is the output of the following code?

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b, a is b)
```

A. True True

B. True False '

C. False False

D. False True

Explanation: == compares values (equal contents !' True). is compares identity; a and b are two distinct list objects, so a is b is False.

COMPREHENSIONS & GENERATORS

Q26. What is the output of the following code?

```
print([x for x in range(5) if x % 2 == 0])
```

A. [0, 2, 4] '

B. [1, 3]

C. [0, 1, 2, 3, 4]

D. [2, 4]

> Explanation: The comprehension keeps only even numbers from range(5): 0, 2, and 4.

FUNCTIONS & SCOPE

Q27. What is the output of the following code?

```
def f(a, b=2, *, c):
    return a + b + c

print(f(1, c=3))
```

A. 6 '

B. Error

C. 4

D. 1

> Explanation: The * marks c as keyword-only. Calling f(1, c=3) gives a=1, b=2 (default), c=3, so the sum is 6.

OOP

Q28. What is the output of the following code?

```
class C:
    @classmethod
    def create(cls):
        return cls.__name__

class D(C):
    pass

print(D.create())
```

A. C

B. D '

C. Error

D. cls

> Explanation: In a classmethod, cls is bound to the class it is called on. D.create() passes D as cls, so cls.__name__ is 'D'.

OOP

Q29. What is the output of the following code?

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

print(MathUtils.add(2, 3))
```

A. 5 '

B. Error

C. 23

D. None

Explanation: A staticmethod receives no implicit self/cls and can be called directly on the class. add(2, 3) returns 5.

EXCEPTION HANDLING

Q30. What is the output of the following code?

```
try:
    raise ValueError("bad")
except ValueError as e:
    print(e)
```

A. ValueError

B. bad '

C. Error

D. ValueError: bad

Explanation: Printing an exception instance shows its message argument. The raised ValueError carries "bad", so that is printed.

STRINGS

Q31. What is the output of the following code?

```
print("a,b,c".split(","))
```

A. ['a', 'b', 'c'] '

B. 'a b c'

C. ['a,b,c']

D. Error

Explanation: str.split(',') splits the string at each comma and returns a list of the resulting substrings: ['a', 'b', 'c'].

STRINGS

Q32. What is the output of the following code?

```
print("-".join(["1", "2", "3"]))
```

A. 1-2-3 '

B. ['1', '2', '3']

C. 123

D. Error

Explanation: str.join() concatenates the iterable's items using the string as a separator, producing "1-2-3".

DATA TYPES

Q33. What is the output of the following code?

```
print(2 ** 3 ** 2)
```

A. 64

B. 512 '

C. Error

D. 256

Explanation: The ** operator is right-associative, so this is 2 ** (3 ** 2) = 2 ** 9 = 512, not (2 ** 3) ** 2.

LISTS & TUPLES

Q34. What is the output of the following code?

```
a = [3, 1, 2]
b = a.sort()
print(b)
```

A. [1, 2, 3]

B. None '

C. [3, 1, 2]

D. Error

Explanation: list.sort() sorts the list in place and returns None. So b is None. (Use sorted(a) to get a new sorted list.)

DICTIONARIES & SETS

Q35. What is the output of the following code?

```
d = {'x': 1, 'y': 2}
for k in d:
    print(k, end=" ")
```

A. 1 2

B. x y '

C. ('x', 1) ('y', 2)

D. Error

Explanation: Iterating over a dictionary yields its keys by default, so this prints 'x y '.

DICTIONARIES & SETS

Q36. What is the output of the following code?

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a & b)
```

A. {1, 2, 3, 4}

B. {2, 3} '

C. {1, 4}

D. {1}

Explanation: The & operator computes the set intersection — elements present in both sets — which is {2, 3}.

COMPREHENSIONS & GENERATORS

Q37. What is the output of the following code?

```
g = (x for x in range(3))
print(list(g))
print(list(g))
```

A. [0, 1, 2]
[0, 1, 2]

B. [0, 1, 2]
[] '

C. []
[]

D. Error

Explanation: A generator can be consumed only once. The first list(g) exhausts it to [0, 1, 2]; the second finds nothing left and returns [].

DECORATORS & CLOSURES

Q38. What is the output of the following code?

```
def deco(f):
    def wrapper(*args):
        return f(*args) + 1
    return wrapper

@deco
def add(a, b):
    return a + b

print(add(2, 3))
```

A. 5

B. 6 '

C. Error

D. 23

Explanation: wrapper forwards its arguments to the original add (2 + 3 = 5) and adds 1, returning 6.

OOP

Q39. What is the output of the following code?

```
class A:
    def greet(self):
        return "A"

class B(A):
    def greet(self):
        return "B" + super().greet()

print(B().greet())
```

A. B

B. BA '

C. AB

D. A

Explanation: B.greet() returns "B" concatenated with super().greet(), which calls A.greet() returning "A". The result is "BA".

DATA TYPES

Q40. What is the output of the following code?

```
print(bool(""), bool("0"), bool([]), bool([0]))
```

A. False False False False

B. False True False True '

C. True True True True

D. False True True True

Explanation: An empty string and empty list are falsy. A non-empty string "0" and a non-empty list [0] are truthy (content value is irrelevant). So the result is False True False True.
